Int J Softw Tools Technol Transfer (2010) 12:263–271
DOI 10.1007/s10009-010-0148-8

GRABATS 2008

GrGen.NET

The expressive, convenient and fast graph rewrite system

Edgar Jakumeit · Sebastian Buchwald · Moritz Kroll

Published online: 28 April 2010
© Springer-Verlag 2010

Abstract GrGen.NET is a generative programming system for graph rewriting, transforming intuitive and expressive rewrite rule specifications into highly efficient .NET code. The user is supported by a convenient environment consisting of a graph viewer, an interactive shell with integrated debugging support, and an elegant domain-specific language for the combination of rewrite rules. After rapid prototyping with these tools, the resulting graph transformation programmes can be easily integrated into arbitrary .NET applications to serve as the algorithmic kernel. Expressiveness, convenience, and speed are exemplified by GrGen-solutions to the case studies AntWorld, Refactoring, and Conference Scheduling—besides others.

Keywords General purpose graph transformation · Graph rewriting · Domain-specific language · Generative programming tool · Search-plan-driven graph pattern matching

1 Introduction

In this chapter we will give an introduction into GrGen.NET [2,9,15] and its specification languages, which will serve as a basis to the following chapters in which we will present our solutions to the GraBaTs 08 [23] challenges Conference Scheduling, AntWorld, and Programme Graphs. Afterwards, we will discuss our experience with GraBaTs, firstly regarding new features learned, secondly casting a recurring discussion into paper form, before we finally conclude.

E. Jakumeit (✉) · S. Buchwald · M. Kroll
Institut für Programmstrukturen und Datenorganisation,
Universität Karlsruhe, Karlsruhe, Germany
e-mail: grgen@ipd.info.uni-karlsruhe.de
URL: http://www.grgen.net

The application domain neutral graph rewrite system GrGen.NET licensed under LGPL3 [16] is combined from two groups of components, the first being the graph rewrite generator GrGen itself together with the execution environment libGr offering the basic functionality of the system, and the second being the interactive command line GrShell plus the graph viewer yComp, offering a rapid prototyping environment.

The base system

The graph rewrite generator reads in files containing specifications of the *graph model* and of the *graph rewrite rules* to be used and emits highly efficient C#-programmes / .NET-assemblies implementing these specifications, assisted by the runtime library, which in addition offers a *control language for rule application*.

The graph model language

The graph model language allows the specification of typed and attributed multigraphs with multiple inheritance on node and edge types, offering directed as well as undirected edges. Types bring safety and performance with them, while multiple inheritance with the root types `Node` and `Edge` allows for concise matching patterns. Graph elements are attributed depending on their type with values of the basic attribute types `boolean`, `int`, `float`, `double`, `string`, `object`, or user-defined `enums`, plus `set<T>` and `map<S,T>` with `S` and `T` being basic attribute types (the latter have been added to better support users from the area of computer linguistics needing string dictionaries). In addition, the graph may be validated against connection assertions constraining the degrees and types of edges between nodes of certain types.

The pattern language

The graph rewrite language offers `tests` consisting only of a pattern to be matched, and `rules` consisting of a pattern part to be matched and a rewrite part to be applied.

The pattern part is built from node and edge declarations or references with an intuitive and easy to learn syntax: Nodes are declared by `n:t`, where `n` is an optional node identifier, and `t` its type, a dot `.` is used for introducing an anonymous node of type `Node`. An edge `e` with source `x` and target `y` is declared by `x -e:t-> y`, whereas `->` introduces an anonymous edge of type `Edge`. Nodes and edges are referenced outside their declaration by `n` and `-e->`, respectively. Undirected edges are declared by `x -e:t- y` or `-` and are referenced by `-e-`. Furthermore, there are:

- Negative Application Conditions (NACs [7]), which are pattern graphs which must not be present in the host graph for the rule to be applicable; they are specified in `negative` blocks and may be arbitrarily nested.
- Type and attribute conditions, given within an `if`-clause, constraining the allowed types and attribute values of matched elements by numerous type and value expressions consisting of boolean, arithmetic, string, set, and map operations.
- Homomorphic matching in addition to the default isomorphic matching, allowing distinct pattern elements given within the `hom()`-statement to be bound to the same host graph element.
- Parameters allowing pattern elements to be preset from outside the pattern before the rest of the pattern is searched.
- Several more to be introduced later on.

The rewrite language

The relationship between the pattern and the rewrite part is given by a preservation morphism according to the theoretically well-founded SPO [7] approach, while the DPO [5] approach is supported as well (by prepending `dpo` before `rule` in the specification). Syntactically, it is specified by a `modify`- or `replace`-block nested within the pattern. With `replace`-mode, graph elements which are referenced within the replace-block are kept, graph elements declared in the replace-block are created, and graph elements declared in the pattern, not referenced in the replace-part are deleted. With `modify`-mode, all graph elements are kept, unless they are specified to be deleted within a `delete()`-statement. Additionally allowed rewrite constructs are:

- Attribute recalculations: results of expressions can be assigned to the attributes of matched or new elements within an `eval`-statement.
- Retyping: matched elements can be retyped not necessarily respecting the type hierarchy—keeping common attributes of initial and final type. This comes in handy if incident edges are to be kept.
- Creation with dynamic types: new graph elements may be created using the actual type of some matched element (more exact than the specified static supertype).
- Element returns: non-deleted elements can be returned to users outside the rule.

The rule application control language

Graph rewrite sequences are a domain-specific language used to combine single rule/test applications into graph rewriting (sub-)programmes. They resemble a union of logical and regular expressions.

- A rule application $(y_1, \ldots y_k) = r(x_1, \ldots, x_m)$ attempts to extend the matches of its parameters $x_1, \ldots, x_m$ to an arbitrary match of its pattern so that the rule can be applied. If this is possible, the application *succeeds*, and defines the variables $y_1, \ldots y_k$; otherwise it *fails*. A test is handled the same way, but does not modify the graph.
- For rewrite sequences S_1, S_2, the logical operations conjunction S_1 && S_2 and disjunction S_1 || S_2 are evaluated lazily from left to right: S_2 is not evaluated if the success or failure of S_1 does already determine the result of the operation. Their strict counterparts & and | always evaluating the second operand exist as well, supplemented by the negation ! .
- Iteration is supported by the construct $S[n : m]$ with the rewrite sequence S and the non-negative integers n and m, which executes S until it fails, but at most m-times. $S[n : m]$ fails, if S was not executed at least n-times. Typical iterations can be abbreviated with S^* ($n = 0$, $m = \infty$) and S^+ ($n = 1$, $m = \infty$).
- Furthermore, variables may be declared, in order to store results of rule applications and pass them as arguments to following rule applications.

Graph models, graph rewrite rules and graph rewrite sequences are accessible from any .NET-language via an API (offered by libGr). An application built on top of this API is the GrShell shipped with GrGen.NET.

The rapid prototyping environment

On top of the graph rewrite functionality, the GrGen.NET system offers tools for rapid development and graphical debugging. These are the GrShell, offering interactive or file-based execution of a graph rewrite script, including step-by-step execution of graph rewrite sequences, aided by the

yFiles [34]-based graph viewer yComp, which can visualise the current graph and each rule application, thus finally the entire rewrite process.

Now that we know the basics of the GrGen.NET system, we will have a deeper look at the GrGen solutions to the online and offline challenges posed at the GraBaTs 2008 tool contest.

2 Conference scheduling

The purpose of the Conference Scheduling case study—mapping paper presentations to time slots—was to evaluate the tools of the participating groups under time pressure in form of a live contest. So, rapid prototyping was the key to win the contest. With GrGen.NET we not only managed to be the first to solve the basic assignment, but were also the only ones with a correct solution.

The important features of GrGen.NET which enabled us to solve the problem so quickly are: intuitive, well-designed languages, the capability to find all matches for a pattern, and the ability to execute a graph rewrite sequence for each found match. So in a first step we could look for a rough mapping of papers to consecutive time-slots (13,824 possibilities), while the additional constraints were checked in a second step leading to the 3,008 valid solutions, which were written to a file. Just mapping the papers to any time-slots (as done by some other groups) would have resulted in 10,077,696 possibilities to be thoroughly checked, so the rough mapping already saves much time.

The additional constraints of the basic assignment were easy to express:

```
test checkConstr
{
    // No simultaneous presentations by same person
    negative {
        ps:Person -:presents-> :Paper -:assocTimepos-> t:Slot;
        ps -:presents-> :Paper -:assocTimepos-> t;
    }

    // No presenter is chairing another session simultaneously
    negative {
        ps:Person -:presents-> p1:Paper -:assocTimepos-> t:Slot;
        p1 -:presentedIn-> session:Session;
        ps -:chair-> othersession:Session <-:presentedIn-
            p2:Paper -:assocTimepos-> t;
    }
}
```

The above listing declares checkConstr as a test, which consists of two negative application conditions (NACs) for the constraints. The test fails if one of the NACs matches. The first NAC matches, if one person ps presents two different papers during the same time-slot t; the second NAC matches, if one person ps presents a paper in one session during a time-slot t and chairs another session at the same time.

3 AntWorld

The AntWorld case study is designed to compare the performance of various graph transformation tools. It consists of simple rules, which simulate the life of an ant colony. To achieve good performance many solutions submitted to GraBaTs'08 use a more complex approach than described by the case study itself. As a result, more than half of the submitted solutions contained bugs. This shows, that the real challenge of the AntWorld case study is to develop a fast *and correct* solution, i.e. to find a good balance between performance and simplicity.

GrGen.NET faces this challenge by providing a clean and expressive rule specification language, as well as fully automatic optimisation of transformation rules. It allows to write the required transformation rules straight from the specification, e.g. the excerpt "If the ant is in search mode and no outer neighbour has sufficient pheromones, the ant moves to any of its neighbour fields based on a fair random choice. (However, an ant without food shall not enter the ant hill.)" leads to the following transformation rule for such an ant:

```
rule SearchAimless(curAnt:Ant)
{
    // get position of current ant
    curAnt -oldPos:AntPosition-> old:GridNode;

    // get neighbour field
    old <-:GridEdge-> new:GridNode\AntHill;

    modify {
        // move ant to neighbour field
        delete(oldPos);
        curAnt -:AntPosition-> new;
    }
}
```

The tricky part of the rule is to find all possible neighbours, regardless of whether the neighbour is connected by an incoming or outgoing edge. For this purpose GrGen.NET offers arbitrary directed edges "<-->" matching a directed edge either way, which is one of many features that keep GrGen.NET rules simple and intuitive. As already mentioned above, this simplicity allows a faster and less error-prone development of solutions.

In spite the burden of offering a convenient language, the AntWorld case study reaffirms that GrGen.NET is among the fastest graph transformation tools (cf. [11,28]). This is

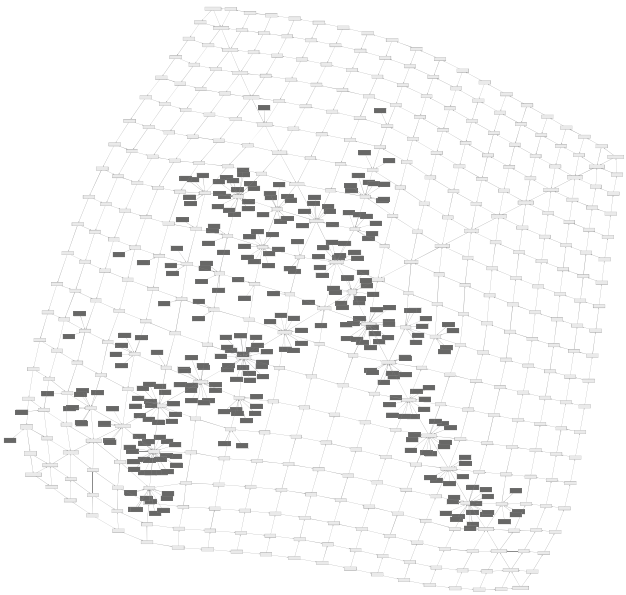

Fig. 1 An AntWorld before grid extension after 61 rounds

due to the generative approach used by GrGen.NET (something we have in common with e.g. the tools PROGRES [26], FUJABA [21], GReAT [14] or recently VMTS [17], contrasted by interpreted or constraint-solving based tools like VIATRA2 [29] or AGG [27]), and the rule-specific search plans [1,33], which determine the order of matching graph elements, allowing a fast pruning of the search space taking dynamic host graph statistics into account. GrGen.NET computes the search plans fully automatically (and the computed search plans perform well, cf. [4]), in contrast to e.g. the semi-automatic visual programming tool FUJABA [21], which for example requires the user to choose the starting point of the search, where a point not chosen wisely may cause a performance penalty of a complexity class; this may happen easily for users not knowing the implementation, as witnessed with the Varró-Benchmark [11,32].

Using GrShell and yComp, it was very easy to play around with the rules and see what they actually do (how and when they get applied, what they match and what they modify), or, at the abstraction level of the challenge, see the ants crawling (Fig. 1 shows an example state reached while simulating the AntWorld).

4 Programme graphs

The programme graphs challenge [22] is designed to compare graph rewrite systems regarding their expressiveness, and in addition their ability to interact with the user. It requires the capability to import and export GXL [13,35]. The task is to implement the refactorings encapsulate field and move method on a given graph representation of programmes (especially a given host graph). In the following we will introduce several features of GrGen with relevance to this case.

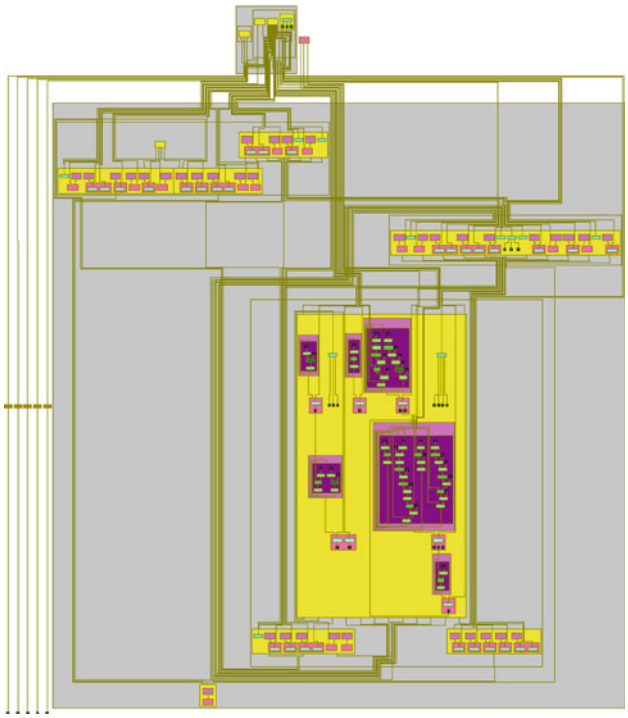

Fig. 2 Overview of the initial programme graph

Graph relabelling

GrGen.NET is not only a graph rewriting system, it is also a graph relabelling system [18]. With the syntax `y:t<x>` you may specify a graph element `y` to be a retyped (to the new type `t`) version of an original graph element `x` of a different—not necessarily—super type. This allows to keep the incident edges of a node untouched, while retyping it to its new type. This is a feature which was included in GrGen because of its usefulness in optimisations on graph-based compiler intermediate representations of programmes in SSA form, which was the original task of the GrGen graph rewrite system. It is used in the encapsulate field refactoring of the programme graphs challenge to retype the `Access` or `Update` nodes into `call` nodes, keeping all the users of the original node, thus saving us from iterating through this unknown set in order to replicate the incident edges which would be required, if we had to delete the original node and create a new `call` node.

Nested graph layout

Graphs allow fast visual understanding—but only as long as the amount of nodes and edges is kept low; the given sample

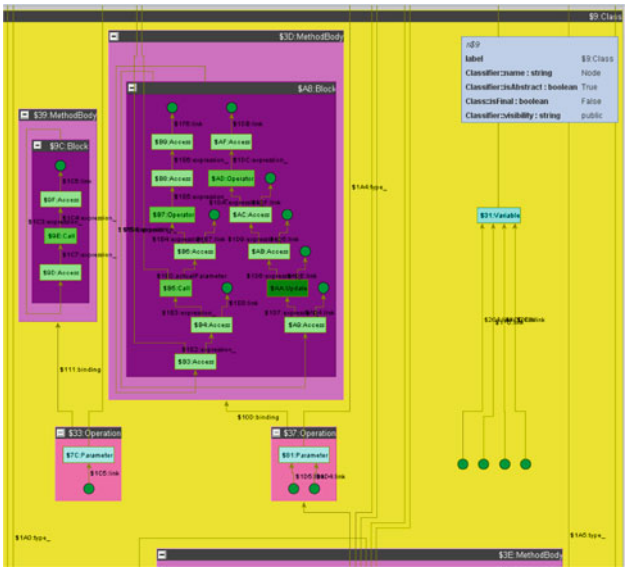

Fig. 3 Some details of the "Node" class

graph (depicted in Figs. 2 and 3) with its more than 500 graph elements is beyond comprehension if presented in a flat layout. Luckily, a lot of graphs are hierarchically structured, with the programme graphs or compiler intermediate graphs falling into this category. For them our graph viewer offers the layout Hierarchic, extending nodes to large areas containing entire subgraphs, and the layout Compilergraph, developed out of the needs of day-to-day work with a compiler intermediate representation as an extension to Hierarchic, cutting certain edges, marking the start and endpoint of the cut with fat dots, allowing to jump to the other point by clicking on either one. Together with colour coding (grey for packages, yellow for classes, magenta for methods, cyan for variables and green for expressions), this allows for an understandable, well-structured visualisation of the host graph, in contrast to the unusable haystack of a flat layout.

The layout as well as the colours are configured by commands in the graph rewrite script of our solution; further layouts are available, the same known from yFiles which yComp uses.

Emit and exec

GXL file import is achieved by a specialised programme `gxl2grs`, transforming the given (invalid) .gxl into an equivalent .grs. Exporting is done by normal GrGen rules, as given in the file `JavaProgramGraphsGxlDumper.gri` (within the `examples/JavaProgramGraphs`-directory of the GrGen.NET package).

Features used in the export process are:

- the `emit` statement, which allows to emit a sequence of string expressions to stdout or into an arbitrary file as specified by the user,
- the `exec` statement, which allows to execute a graph rewrite sequence directly after the basic rule was applied, with the graph elements of the pattern and rewrite part being available as arguments to the tests and rules in the rewrite sequence,
- the visited flags, which allow to mark elements, in order to highlight them for further processing or to prevent them from being processed twice during a graph walk; they could be introduced manually in the graph model, but this specialised treatment is more convenient to the user and allows for a more efficient implementation (as they are saved in some excess bits of the graph elements).

One of the rules used iteratively for exporting the graph is shown below, matching a package node not yet visited, marking it as visited and emitting the node with its internal name plus the name attribute.

```
rule dumpNodePackage
{
    n:Package;

    if { !visited(n); }

    modify {
        eval { visited(n) = true; }

        emit("<node id=\"", nameof(n), "\">",
            "<type xlink:href=\"Package\"/>",
            "<attr name=\"name\"> <string>",
                n.name, "</string> </attr>",
            "</node>");
    }
}
```

The same mechanism is used in the MOF-Suite for GrGen.NET, extending GrGen for model transformation [10], capable of importing and exporting UML models conforming to MOF 2.0 given in XMI.

Recursive patterns

The Java programme graphs defined in the challenge are to a good degree tree-like, resembling an abstract syntax tree with additional intermediate edges from uses to definitions. Tree-like or other structures of recursive nature can get matched or even rewritten neatly by one application of a single rule. This is allowed by the concept of *subpatterns* respectively *subrules* (which in our case are subpatterns with nested `modify/replace`-parts), combined with

alternative parts, as introduced in [12] (based on Hyperedge Graph Replacement [6] on the meta level to assemble rules to be applied on the object level). Similar approaches—limited to recursive subpatterns—can be found in the tools VIATRA2 [31] and TEFKAT [19]. In [12] sub*rules* were used to model the transcription of a DNA strand into an RNA strand; here sub*patterns* are used to model an iterated path condition in the implementation of the "Move Method Refactoring", checking whether the name of the operation already exists in the scope of the target class. This check is accomplished by the following subpattern `methodNameExistsSuper`, which succeeds if in the given class `cls` there is an operation `op`, which either bears the name we are searching for (alternative case "here"), or there is a superclass `super`, in which the check succeeds (alternative case "super").

```
pattern methodNameExistsSuper(opForName:Operation, cls:Class)
{
    cls <-:belongsTo- op:Operation;

    alternative {
        here {
            if { op.name==opForName.name; }
        }
        super {
            cls -:extends_-> super:Class;
            :methodNameExistsSuper(opForName, super);
        }
    }
}
```

(Furthermore, the subpatterns are used for dumping the graph to a text file for debugging, obeying the nesting structure; there the `emitpre` and `emitpost` statements are used, which allow to emit text before and after emitting the text for the nested subpatterns.)

5 Features learned

The GraBaTs—Graph-Based Tools Contest—is designed to stimulate learning own limitations and learning from other tools, as well as serving as a discussion platform for tool creators and potential users. After a short explanation of the development goals of GrGen.NET we will see how feedback from the tool contest helped improving the GrGen tool regarding each one of them. In the following Sect. 6 we want to cast a recurring discussion of GraBaTs about textual and graphical representations into paper form.

Development goals

Based on our own experience of day-to-day work with rewriting of a graph-based compiler intermediate language—a real world task tackled with graph rewriting—we focussed on

- expressive language constructs allowing for a concise and descriptive solution
- high execution speed at modest memory consumption, so that you can really use the result of your work
- convenient development, realised by stepwise and graphical debugging, together with a well-designed API and an extensive user manual.

The tool contest gave valuable input on each of these fields, based on it we are currently designing and implementing the following extensions to the GrGen.NET system. Most of them will be available in the next major release.

Convenience: GXL

The current solution with the gxl2grs converter and the user-specified export is working, but lacks convenience. Instead we will add an importer and an exporter for GXL 1.0 directly to the libGr, available from the API and the GrShell. This will ease tool comparisons and migration.

Execution speed: storages

Storages—sets of nodes—gave the tool VMTS the thrust to win the performance case AntWorld (cf. [20]). In GrGen.NET they will be represented as variables of type `set<Node>`, accessible for graph rewrite sequences like `s.add(el)`, adding an element to a set, `s.rem(el)` for removing an element from a set, and the iteration `for{x in s; r(x)}` executing the nested sequence for all elements in the set, with a new element bound to the iteration variable in each step. Looking up elements in the storages instead of searching for marked elements in the graph will allow for faster solutions. (But sets of nodes will not be made available as attribute types in the graph model because we regard this usage to be too error-prone).

Expressiveness: pattern cardinality and PAC

The refactoring case indeed proved to be a stress test regarding expressiveness. While the recursive patterns/rules were a major leap ahead, they were not as useful in solving this challenge as expected. While they do well in matching structures of arbitrary depth, as e.g. iterated paths, or of arbitrary breadth, e.g. a multinode, they have their weaknesses in matching recursive structures of arbitrary depth *and* arbitrary breadth, like an abstract syntax tree. To overcome this limitation they will be supplemented by an `iterated`-construct, matching its nested pattern as often as possible. With this feature, it will be possible to match a spanning tree as simple as this:

```
pattern SpanningTree(root:Node)
{
    iterated {
        root -- next:Node;
        :SpanningTree(next);
    }
}
```

A further problem we experienced in this case was the incapability to check an iterated path condition targeting an unknown but already matched node (matched in a subpattern containing the current pattern). The check always fails because the node is locked due to the isomorphy constraint. This problem will be solved by Positive Application Condition patterns—syntactically specified by an `independent`-block, requiring the nested pattern to get matched, but not constraining the contained elements to get matched isomorphically to the elements of the nesting pattern. So in the following test, there will be always a match of the iterated path heading to a node of type `A` in case the test pattern was found—the elements of the test pattern once again.

```
test someTest
{
    na:A<-- . <-- n:Node;
    independent {
        :IteratedPathToA(n);
    }
}
```

6 Discussion

As elaborated at the beginning of the previous section, our major development goals were:

- language expressiveness
- execution speed
- development convenience

We did *not* invest into the development of graphical languages and graphical editors. Regarding several other tools offering graphical languages and graphical editors (FUJABA [8], VMTS [17], AGG [27]) and the discussions on the "graphical versus textual" issue during the tool contests, we think this is a good place to cast our arguments (largely in favour of textual languages) into paper form.

Benefits

The superiority of graphical representations and editors or textual representations and editors depends on the domain and is to a certain degree a matter of taste. For rule application control and attribute computations we doubt graphical representations and editors to be superior at all, a claim we think is backed by the unsuccessfulness of graphical programming languages as compared to textual ones. For graph pattern matching and rewriting on the other hand a graphical representation would be a more intuitive and direct user interface—at least as long as the graphs stay small, they quickly lose the property of fast visual understandability with an increase in the number of elements. But the less direct textual representations offer some other benefits, and most important in our case, are less expensive.

Costs

Given limited resources, you must choose into which areas to invest. While graphical representations are the more intuitive ones for the user in the case of graph patterns, they come at a higher cost. As they are not offering a higher expressiveness, only a more convenient user interface, we decided to follow a first-things-first policy and focussed on the aforementioned development goals. In our opinion, a textual language which offers the user the expressiveness and performance he needs to tackle a given problem is more important than a graphical language of less expressiveness backed by a slower engine developed given the same resources.

Tool support

The lower costs are to a certain degree based on tool support, with compiler construction tools like parser generators—which gave us for instance the ability to add the `$`-operator for indeterministic choice needed for the AntWorld-case within the shortest amount of time—being more mature than graphical representation toolkits. But while this gap will narrow in the future, we doubt it will vanish, since graphical representations are the more complex technology compared with textual representations.

Robustness

Technological complexity has another recurrence: human optimised textual representations allow the user to read, edit, and write the entities of design with the simplest of editors, in contrast to graphical representations which require the original viewer/editor to be available and running without major bugs. We think that the more complex technology should be built on top of the simple one: a graphical rule editor reading and writing the textual representation can be implemented later on as an add-on, even without contact to the original developers, so the development process itself gets more robust. (Software developers willing to implement a graphical rule editor on top of GrGen.NET are welcome.)

Harnessing generative power

Finally, domain-specific languages mapping to the graph rewrite language or simple helper programmes generating rules programmatically (exploiting similarities) can be implemented and debugged easier. Generation is just a matter of filling some schemes with variables and invoking a text emit. Further on, a compiler operating on a human-optimised format, itself meant to be used by humans, normally gives useful error messages, which, together with the fact that all information is immediately available in the file processed, allows locating the error quickly. (And our experience of implementing the GrGen.NET code generator as well as an optimiser generator for a compiler intermediate language emitting GrGen rules taught us that this is a point not to be neglected.) A graphical representation on the other hand must be accessed via the API of the editor or via the machine optimised import/export format, which will be cumbersome or impossible to read directly; and the tool normally will give you only the information that import failed, as it is only meant to be used for reading/writing its own rules, with errors handled by the developers in their debugger.

7 Conclusion

GrGen.NET is a special purpose software development system and a general purpose graph rewrite system. It is designed for software developers and meant to be used for the generation of the algorithmic core of applications processing graph structured data. Its user interface and implementation were chosen to perform reasonably well on any graph rewriting task—and we think it does so, looking at uses in e.g. compiler construction [24], model transformation [10], computer linguistics [3] or computational biology [25].

Regarding the GraBaTs challenges, GrGen.NET has shown its usability and superiority for rapid prototyping in the live contest Conference Scheduling case (as well as in the GraBaTs 09 live contest Conveyor Belt case [30]). Normally fast speed of development leads to low execution speed of the resulting code, which is not the case with GrGen.NET, as witnessed in the AntWorld challenge or proven by the Varró benchmark [32]. This is due to the generative approach and the host graph sensitive search plan based graph pattern matching, which allows the user to write down directly what he thinks, still getting a high-performance solution. With the `iterated` language construct introduced due to feedback from the Refactoring case GrGen.NET has taken a leading position in rewrite rule expressiveness, now being capable of handling recursive structures of arbitrary depth and breadth, which allows for further shifting of work from the imperative control programmes to the declarative rewrite rules.

GrGen.NET (available at www.grgen.net) offers a convenient development environment consisting of a sophisticated graph viewer and an interactive shell supporting graphical and step-wise debugging; it provides intuitive and very expressive specification languages and a highly optimised code generator, yielding the highest combined speed of development and execution.

Acknowledgments We thank our three reviewers, the two anonymous ones as well as our third one, Rubino Geiß.

References

1. Batz, G.V.: An optimization technique for subgraph matching strategies. Internal report. http://www.info.uni-karlsruhe.de/papers/TR_2006_7.pdf (2006)
2. Blomer, J., Geiß, R.: The GrGen.NET user manual. Internal report. http://www.info.uni-karlsruhe.de/software/grgen/GrGenNET-Manual.pdf. Version: 2008
3. Bédaride, P., Gardent, C.: Semantic normalisation: a framework and an experiment. In: Eighth International Conference on Computational Semantics (2009)
4. Batz, G.V., Kroll, M., Geiß, R.: A first experimental evaluation of search plan driven graph pattern matching. In: Applications of Graph Transformation with Industrial Relevance (AGTIVE '07) Proceedings. http://www.info.uni-karlsruhe.de/papers/agtive_2007_search_plan.pdf (2008)
5. Corradini, A., Montanari, U., Rossi, F., Ehrig, H., Heckel, R., Löwe, M.: Algebraic approaches to graph transformation. Part I: basis concepts and double pushout approach. In: Handbook of Graph Grammars and Computing by Graph Transformation, vol. 1 (1997)
6. Drewes, F., Kreowski, H.J., Habel, A.: Hyperedge replacement graph grammars. In: Handbook of Graph Grammars and Computing by Graph Transformation, vol. 1 (1997)
7. Ehrig, H., Heckel, R., Korff, M., Löwe, M., Ribeiro, L., Wagner, A.: Algebraic approaches to graph transformation. Part II: Single Pushout Approach and Comparison with Double Pushout Approach. In: Handbook of Graph Grammars and Computing by Graph Transformation, vol. 1 (1997)
8. Fischer, T., Niere, J., Torunski, L., Zündorf, A.: Story diagrams: a new graph grammar language based on the unified modelling language and java. In: Theory and Application of Graph Transformations, pp. 157–167 (2000)
9. Geiß, R., Batz, G.V., Grund, D., Hack, S., Szalkowski, A.M.: GrGen: a fast SPO-based graph rewriting tool. In: Proceedings of 3rd International Conference on Graph Transformation (ICGT '06). http://www.info.uni-karlsruhe.de/papers/grgen_icgt2006.pdf (2006)
10. Gelhausen, T., Derre, B., Geiß, R.: Customizing GrGen.NET for model transformation. In: GraMoT, pp. 17–24 (2008)
11. Geiß, R., Kroll, M.: On Improvements of the Varró benchmark for graph transformation tools. Internal report. http://www.info.uni-karlsruhe.de/papers/TR_2007_7.pdf. Version: 2007
12. Hoffmann, B., Jakumeit, E., Geiß, R.: Graph rewrite rules with structural recursion. In: 2nd International Workshop on Graph Computational Models (GCM 2008). http://www.info.uni-karlsruhe.de/papers/GCM2008.pdf (2008)
13. Holt, R.C., Schürr, A., Elliott Sim, S., Winter, A.: GXL: a graph-based standard exchange format for reengineering. In: Science of Computer Programming (2005)

14. Karsai, G., Agrawal, A., Shi, F., Sprinkle, J.: On the use of graph transformation in the formal specification of model interpreters. J. Univers. Comput. Sci. **9**, 1296–1321 (2003)
15. Kroll, M., Geiß, R.: Developing graph transformations with GrGen.NET. Internal report. http://www.info.uni-karlsruhe.de/papers/agtive_2007_grgennet.pdf. Version: 2007
16. GNU Lesser General Public License. http://www.gnu.org/licenses/lgpl-3.0.txt. Version: 2007
17. Levendovszky, T., Lengyel, L., Mezei, G., Charaf, H.: A systematic approach to metamodeling environments and model transformation systems in VMTS. Electron. Notes Theor. Comput. Sci. **127**, 65–75 (2005)
18. Litovsky, I., Métivier, Y., Sopena, E.: Graph relabelling systems and distributed algorithms. In: Handbook of Graph Grammars and Computing by Graph Transformation, vol. 3. World Scientific Publishing, Singapore (1999)
19. Lawley, M., Steel, J.: Practical declarative model transformation with Tefkat. In: MoDELS Satellite Events, pp. 139–150 (2005)
20. Mészáros, T., Mezei, G., Levendovszky, T., Asztalos, M.: Manual and automated performance optimization of model transformation systems. J. Softw. Tools. Technol. Transfer (2010). doi:10.1007/s10009-010-0151-0
21. Nickel, U., Niere, J., Zündorf, A.: The FUJABA environment. In: ICSE '00 Proceedings of the 22nd International Conference on Software Engineering, pp. 742–745 (2000)
22. Pérez, J., Crespo, Y., Hoffmann, B., Mens, T.: A case study to evaluate the suitability of graph transformation tools for program refactoring. Int. J. Softw. Tools. Technol. Transfer (2010). doi:10.1007/s10009-010-0153-y
23. Rensink, A., Van Gorp, P.: Graph-based tools: the contest. In: Proceedings 4th International Conference on Graph Transformation (ICGT '08) (2008)
24. Schösser, A., Geiß, R.: Graph rewriting for hardware dependent program optimizations. In: Applications of Graph Transformation with Industrial Relevance (AGTIVE '07) Proceedings. http://www.info.uni-karlsruhe.de/papers/agtive_2007_firm.pdf. Version: 2007
25. Schimmel, J., Gelhausen, T., Schaefer, C.A.: Gene expression with general purpose graph rewriting systems. In: Proceedings of the 8th GT-VMT Workshop (2009)
26. Schürr, A., Winter, A.J. , Zündorf, A.: The PROGRES approach: language and environment. In: Handbook of Graph Grammars and Computing by Graph Transformation, vol. 2, pp. 487–550 (1999)
27. Taentzer, G.: AGG: a graph transformation environment for modeling and validation of software. In: Applications of Graph Transformations with Industrial Relevance, pp. 446–453 (2004)
28. Taentzer, G., Biermann, E., Bisztray, D., Bohnet, B., Boneva, I., Boronat, A., Geiger, L., Geiß, R., Horvath, Á., Kniemeyer, O., Mens, T., Ness, B., Plump, D., Vajk, T.: Generation of Sierpinski triangles: a case study for graph transformation tools. In: Proceedings of 3rd Interlnational Workshop on Applications of Graph Transformation with Industrial Relevance (AGTIVE '07), pp. 514–539 (2008)
29. Varró, D., Balogh, A.: The model transformation language of the VIATRA2 framework. Sci. Comput. Program. **68**(3), 214–234 (2007)
30. Van Gorp, P., Levendovszky, T., Rensink, A.: Live Challenge Problem. http://is.tm.tue.nl/staff/pvgorp/events/grabats2009/cases/live_problem.pdf. Version: 2009
31. Varró, G., Horváth, Á., Varró, D.: Recursive graph pattern matching. In: Applications of Graph Transformations with Industrial Relevance, pp. 456–470 (2008)
32. Varró, G., Schürr, A., Varró, D.: Benchmarking for graph transformation. In: VLHCC '05: Proceedings of the 2005 IEEE Symposium on Visual Languages and Human-Centric Computing, pp. 79–88 (2005)
33. Varró, G., Varró, D., Friedl, K.: Adaptive graph pattern matching for model transformations using model-sensitive search plans. In: GraMot 2005, International Workshop on Graph and Model Transformations, pp. 191–205 (2006)
34. Wiese, R., Eiglsperger, M., Kaufmann, M.: yFiles: visualization and automatic layout of graphs. In: Graph Drawing, 9th International Symposium (2002)
35. Winter, A., Kullbach, B., Riediger, V.: An overview of the GXL graph exchange language. In: Software Visualization—International Seminar Dagstuhl Castle (2002)